

flowToApexAI

User Guide — AI-powered Flow to Apex migration that runs entirely inside Salesforce using Einstein and Prompt Builder.

1. Overview

flowToApexAI converts Salesforce Record-Triggered Flows into Apex Trigger and Handler code without ever leaving the org. A user types a Flow API name into the Lightning App, clicks Convert, and within seconds a `FlowMigration__c` record appears with the generated Apex code rendered in a code viewer with copy buttons.

Behind the scenes the tool reads the Flow metadata via the Tooling API, sends it to a versioned Prompt Template in Prompt Builder, and persists the Einstein-generated Apex back into a custom object. All data stays inside the Einstein Trust Layer.

2. Prerequisites

- An org with **Einstein Generative AI, Prompt Builder**, and permissions to invoke the Models API.
- A **Remote Site Setting** pointing at the org's My Domain URL (for the Tooling API callout).
- The **flowToApexAI** package deployed (custom object, Apex services, LWC, Lightning App).
- The Lightning App **Flow to Apex AI** made visible to your profile.

3. Daily usage

The end-to-end workflow from a user's perspective:

- **1. Open the App.** App Launcher → **Flow to Apex AI** → the **Flow Converter** tab opens by default.
- **2. Enter the Flow API name.** For example, `Account_Set_Default_Industry`. This is the developer name shown in Setup → Flows.
- **3. Click Convert to Apex.** A spinner shows for 10–30 seconds while the tool reads the Flow, sends it to Einstein, and persists the result.
- **4. Review the generated Apex.** Trigger and Handler appear in tabs directly in the UI with copy buttons. Click **Open Migration Record** to see the same code on the persistent `FlowMigration__c` record.
- **5. Copy or deploy.** Copy the code into VS Code or paste into the Developer Console as a new class. Review the logic and add CRUD / FLS checks before deploying to production.

4. Understanding the output

Every conversion creates a `FlowMigration__c` record. The fields:

Field	Purpose
-------	---------

Migration ID	Auto-numbered identifier (FM-0001, FM-0002, ...)
Source Flow Name	The Flow API name that was migrated
Status	New / In Progress / Generated / Failed
Generated Apex	The Trigger + Handler code, shown in the code viewer
Error Message	Populated only when Status = Failed

The *Flow Migration Code Viewer* component on the record page shows Trigger and Handler in styled tabs with copy buttons, so reviewers can read and copy the code without scrolling through a raw text area.

5. Architecture (one-pager)

```
User
  v (types Flow API name)
[ LWC: flowToApexConverter ]
  v (@AuraEnabled)
[ Apex: FlowToApexService.migrate(flowApiName) ]
  v
  +--> FlowReaderService    --> Tooling API (Flow JSON)
  +--> EinsteinPromptClient --> Prompt Builder template -> Einstein
  +--> insert FlowMigration__c with the generated Apex
  v
[ Lightning record page + Code Viewer ]
  v
User reviews and copies the generated Trigger + Handler.
```

6. Troubleshooting

HTTP 401 INVALID_SESSION_ID

Triggered when calling from LWC context. The SessionIdPage Visualforce page handles this by returning a REST-API-valid token. Make sure the page is deployed and the Remote Site Setting points at the My Domain URL.

HTTP 400 / Flow not found

The Flow API name does not match any active Flow in the org. Open Setup → Flows and confirm the developer name matches exactly, including underscores. The Flow must be in Active status, not Draft.

Conversion fails — "missing expected markers"

Einstein returned a free-form response instead of the expected ===TRIGGER=== / ===HANDLER=== format. Re-run the conversion. If it persists, the prompt template may need adjusting in Prompt Builder.

Uncommitted work pending

An older code path tried to DML before completing callouts. The current orchestration does all callouts first and only inserts the FlowMigration record at the end. If you see this, you are running an outdated build — redeploy the latest FlowToApexService.cls.

7. Limits and what to review

- Generated Apex is a **skeleton**. Review logic, add CRUD / FLS checks (WITH SECURITY_ENFORCED or Security.stripInaccessible()) before deploying.
- Apex test classes are **not generated automatically** in this version. Write tests at 200-record bulk scale.
- **Determinism is not 100%**. The LLM may produce slightly different output across runs. Treat the generated code as a starting point, not a byte-for-byte specification.
- **Out-of-scope Flow elements** (Screen, Subflow, Action Call, Wait) may be approximated or skipped by the LLM. Read the generated header and inspect the output carefully for these patterns.